

DOCUMENTAZIONE TECNICA

Schema del database

Tabelle, colonne, vincoli, indici, migrazioni e dati di riferimento del database guestportal



Campo	Valore
Applicativo	Guest Portal — Cisco Catalyst 9800 Guest Management Desk
Documento	DB — Schema del database
Versione	1.0
Data di emissione	18 settembre 2026
Stato	Emesso
Destinatari	IT Infrastructure, Security, Sviluppo, Service Desk
Classificazione	Uso interno
Repository di riferimento	github.com/ACF-Dompe/Centralino
Revisione del codice	main · f427890

Indice

1	Introduzione.....	3
1.1	Motore e connessione	3
1.2	Convenzioni.....	3
1.3	Due assenze deliberate.....	3
2	Schema entità-relazioni.....	4
2.1	Inventario delle tabelle.....	4
3	Tabelle operative.....	6
3.1	guests	6
3.2	sedi	7
3.3	sync_logs.....	8
4	Tabelle di identità e permessi	10
4.1	app_users	10
4.2	app_user_sedi	11
4.3	breakglass_users.....	11
4.4	session.....	12
5	Tabelle di supporto.....	13
5.1	saml_request_ids.....	13
5.2	sms_config.....	13
5.3	wlc_config (legacy)	13
6	Vincoli e indici — riepilogo	15
7	Migrazioni	16
7.1	Strategia	16
7.2	Esecuzione	16
7.3	Il backfill dei parametri del controller.....	16
7.4	Bootstrap dell'account di emergenza	16
8	Dati di riferimento	18
9	Dati personali e ritenzione.....	19
10	Accessi tipici e prestazioni	20

1 Introduzione

Questo documento descrive lo schema del database del Guest Portal: tabelle, colonne, vincoli, indici, la strategia di migrazione e i dati di riferimento necessari perché l'applicazione sia utilizzabile.

1.1 Motore e connessione

Aspetto	Valore
Motore	PostgreSQL 15 o superiore — Azure Database for PostgreSQL Flexible Server
Database	<code>guestportal_<ambiente></code> (per esempio <code>guestportal_prod</code>)
Driver	<code>pg</code> (node-postgres) con pool gestito dall'applicazione
Autenticazione	Token Entra ID tramite <code>DefaultAzureCredential</code> ; l'utente del database è l'identità gestita del backend. Nessuna password nella stringa di connessione.
Scope del token	<code>https://ossrdbms-aad.database.windows.net/.default</code>
TLS	Attivo in produzione, con verifica del certificato del server
Vita della connessione	Massimo 1800 secondi, inattività 30 secondi — entrambi sotto la validità del token

Tabella 1 — Caratteristiche della connessione al database.

1.2 Convenzioni

- Nomi di tabella e colonna in `snake_case`; il dominio aziendale resta in italiano (`sedi`, `sede_id`).
- I riferimenti temporali sono `TIMESTAMP` senza fuso orario, valorizzati con `NOW()` lato server.
- Gli stati sono stringhe con vincolo `CHECK`, non enumerazioni native: una migrazione idempotente può aggiungere una colonna ma non alterare un tipo enumerato senza rischi.
- Non esistono `DROP` nello schema: le migrazioni sono additive, così un rollback dell'immagine non richiede un rollback dello schema.

1.3 Due assenze deliberate

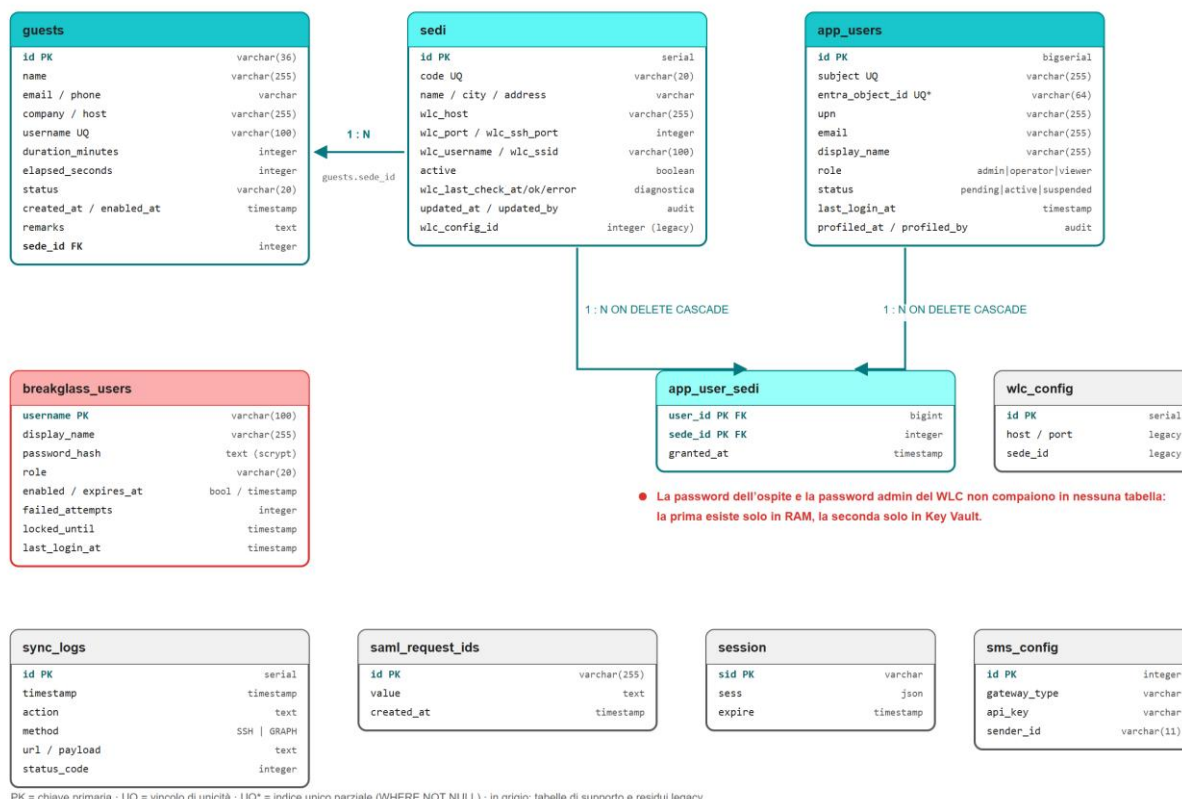
Che cosa non si troverà in questo schema

La password dell'ospite. Non esiste una colonna che la contenga. È generata in memoria, spinta sul controller, consegnata e dimenticata. Non c'è un percorso per rileggerla: la riconsegna avviene per rigenerazione.

La password amministrativa del controller. Vive in Key Vault, una per sede, ed è risolta a runtime dalla variabile d'ambiente `WLC_PASSWORD_<CODICE>`. Nessuna tabella la contiene e nessuna risposta della API la espone: l'unica informazione disponibile è se una password risulti configurata.

2 Schema entità-relazioni

• Schema entità-relazioni — database guestportal



PK = chiave primaria - UQ = vincolo di unicità - UQ* = indice unico parziale (WHERE NOT NULL) - in grigio: tabelle di supporto e residui legacy

Figura 1 — Tabelle del database guestportal, relazioni e raggruppamento funzionale.

2.1 Inventario delle tabelle

Tabella	Gruppo	Righe attese	Ruolo
guests	Operatività	Migliaia	Gli account ospite gestiti dalla console.
sedi	Operatività	Unità	Le sedi e i parametri del rispettivo controller.
sync_logs	Operatività	Decine di migliaia	Storico delle operazioni verso controller e posta.
app_users	Identità	Decine	Directory applicativa popolata al primo accesso SSO.
app_user_sedi	Identità	Centinaia	Sedi concesse a ciascun utente.
breakglass_users	Identità	Unità	Account di accesso di emergenza.
session	Identità	Decine	Sessioni Express persistite (creata da connect-pg-simple).
saml_request_ids	Protocollo	Decine	Identificativi delle AuthnRequest in attesa di risposta.
sms_config	Supporto	1	Configurazione del canale SMS, attualmente dormiente.
wlc_config	Legacy	Unità	Residuo della precedente modellazione, in via di dismissione.

Tabella 2 — *Inventario delle tabelle.*

3 Tabelle operative

3.1 guests

Un ospite, cioè un account temporaneo rilasciato su un controller. La chiave è una stringa generata dall'applicazione nella forma `g-<8 esadecimali>`.

Colonna	Tipo	Vincoli	Descrizione
id	VARCHAR(36)	PK	Identificativo generato dall'applicazione.
name	VARCHAR(255)	NOT NULL	Nome dell'ospite mostrato in console.
email	VARCHAR(255)	—	Indirizzo per la consegna delle credenziali. Assente per gli utenti importati.
phone	VARCHAR(50)	—	Recapito telefonico, previsto per il canale SMS.
company	VARCHAR(255)	DEFAULT 'Ospite Individuale'	Organizzazione di provenienza. Vale Utente WLC per gli importati.
host	VARCHAR(255)	NOT NULL	Referente interno che ospita la visita.
username	VARCHAR(100)	NOT NULL, UNIQUE	Username creato sul controller (g.<slug><3 cifre>).
duration_minutes	INTEGER	NOT NULL DEFAULT 240	Durata concessa. Il server accetta da 1 minuto a 7 giorni.
elapsed_seconds	INTEGER	NOT NULL DEFAULT 0	Tempo trascorso, aggiornato ogni secondo dal servizio timer.
status	VARCHAR(20)	NOT NULL DEFAULT 'pending' CHECK	Uno fra pending, active, expired, deactivated.
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Momento della registrazione.
enabled_at	TIMESTAMP	—	Inizio del conteggio della durata. È da qui che si calcola la scadenza.
remarks	TEXT	—	Annotazione libera; Importato dal WLC per gli account riconciliati.
sede_id	INTEGER	—	Sede di appartenenza. Nullo solo per righe anteriori all'introduzione delle sedi.

Tabella 3 — Colonne della tabella `guests`.

INDICI

```
PRIMARY KEY (id)
UNIQUE      (username)
INDEX idx_guests_status      (status)
INDEX idx_guests_created_at  (created_at DESC)
INDEX idx_guests_sede_id     (sede_id)
```

`idx_guests_sede_id` sostiene la query più frequente dell'applicazione: l'elenco degli ospiti della sede su cui la sessione è posizionata. `idx_guests_created_at` sostiene l'ordinamento predefinito.

Perché non c'è una colonna password

La password in chiaro è generata in memoria, spinta sul controller via SSH, inviata all'ospite e restituita una sola volta all'operatore nella risposta di creazione. Non viene mai scritta. La conseguenza operativa è che la riattivazione di un ospite non può ripristinare la credenziale precedente: occorre usare «Reinvia credenziali», che ne genera una nuova e la spinge sul controller.

NOTA SUGLI STATI

Il vincolo ammette quattro stati, ma il flusso di registrazione crea l'ospite direttamente come **active**: **pending** è previsto dallo schema e non utilizzato dal percorso corrente.

3.2 sedi

Una sede, con i parametri di connessione del proprio controller e la diagnostica dell'ultima verifica. È l'unità di isolamento dei dati: ogni ospite appartiene a una sede e ogni utente è abilitato a un insieme di sedi.

Colonna	Tipo	Vincoli	Descrizione
id	SERIAL	PK	Identificativo numerico.
code	VARCHAR(20)	NOT NULL, UNIQUE	Codice breve (MIL, AQ, NA, TIR, SM). Immutabile : risolve il nome del segreto in Key Vault.
name	VARCHAR(100)	NOT NULL	Denominazione estesa.
city	VARCHAR(100)	NOT NULL	Città.
address	VARCHAR(255)	—	Indirizzo completo.
wlc_host	VARCHAR(255)	—	Indirizzo del controller. Nullo significa «sede creata, controller non ancora configurato» : è uno stato legittimo che il pannello mostra.
wlc_port	INTEGER	NOT NULL DEFAULT 443	Porta HTTPS della interfaccia di amministrazione.
wlc_ssh_port	INTEGER	NOT NULL DEFAULT 22	Porta SSH.
wlc_username	VARCHAR(100)	NOT NULL DEFAULT 'admin_guest'	Account amministrativo sul controller.
wlc_ssid	VARCHAR(100)	NOT NULL DEFAULT 'Dompe Guest'	Nome della rete comunicato all'ospite.
active	BOOLEAN	NOT NULL DEFAULT TRUE	In servizio. Una sede disattivata sparisce dal selettore e dalla sincronizzazione.
wlc_last_check_at	TIMESTAMP	—	Momento dell'ultima verifica di raggiungibilità.
wlc_last_check_ok	BOOLEAN	—	Esito dell'ultima verifica.
wlc_last_check_error	TEXT	—	Messaggio d'errore classificato dell'ultima verifica fallita.
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Ultima modifica.
updated_by	VARCHAR(255)	—	Chi ha effettuato la modifica.
wlc_config_id	INTEGER	—	Residuo legacy del collegamento alla vecchia tabella.

Tabella 4 — Colonne della tabella `sedi`.**INDICI**

```
PRIMARY KEY (id)
UNIQUE      (code)
INDEX idx_sedi_code    (code)
INDEX idx_sedi_active  (active)
```

Perché i parametri del controller stanno qui

Vivevano in una tabella `wlc_config` separata, in relazione uno a uno con `sedi` e collegata da entrambi i lati (`sedi.wlc_config_id` e `wlc_config.sede_id`). I due collegamenti potevano divergere: per questo ogni lettura portava con sé un ripiego e l'unicità dell'associazione era garantita solo «per quanto possibile». Riportare le colonne su `sedi` elimina la classe di errore: una riga per sede, nessun orfano, nessun ripiego.

IL CODICE DI SEDE È IMMUTABILE

`code` determina il nome del segreto in Key Vault e della variabile d'ambiente corrispondente. Cambiarlo scollegherebbe la sede dalla propria password: la API di aggiornamento lo rifiuta. Il vincolo di formato è `^[A-Z0-9]{2,20}$`.

```
code = "MIL" → env WLC_PASSWORD_MIL ← Key Vault WLC-PASSWORD-MIL
```

3.3 sync_logs

Storico delle operazioni eseguite verso i controller e verso il servizio di posta. È la traccia consultabile dalla console e la base per ricostruire che cosa sia successo a un ospite.

Colonna	Tipo	Vincoli	Descrizione
id	SERIAL	PK	Progressivo.
timestamp	TIMESTAMP	NOT NULL DEFAULT NOW()	Momento dell'operazione.
action	TEXT	NOT NULL	Descrizione: <code>create-user g.mrossi417, sync: import ..., resend-email sent (graph), ...</code>
method	VARCHAR(10)	NOT NULL	SSH per i controller, GRAPH per la posta.
url	TEXT	—	Destinazione: <code>host:porta</code> per SSH, indirizzo del destinatario per la posta.
payload	TEXT	—	Dettagli sintetici. I segreti sono oscurati prima della scrittura.
status_code	INTEGER	—	Esito: 201 creato, 200 riuscito, 401 rifiutato, 502 controller non raggiungibile, 0 operazione registrata in modalità non in linea.

Tabella 5 — Colonne della tabella `sync\_logs`.**Oscuramento dei segreti**

Prima di scrivere il `payload` di una creazione utente, la password di destinazione è sostituita da `***`. Era un difetto corretto: finiva in chiaro nello storico, che è consultabile dalla console.

RITENZIONE

La tabella cresce con l'uso: cinque sedi sincronizzate ogni trenta secondi producono circa **14.400 righe al giorno** di sole voci di sincronizzazione. Non esiste una politica di ritenzione automatica; è disponibile

una cancellazione completa riservata al ruolo `admin` (`DELETE /api/sync-logs`). Una politica periodica è raccomandata ed è indicata fra le evoluzioni nel documento HLD.

4 Tabelle di identità e permessi

4.1 app_users

La directory applicativa. Viene riempita **just-in-time** al primo accesso SSO riuscito e poi profilata da un amministratore. Una riga nuova è sempre **viewer** / **pending**: l'utente può autenticarsi ma non agire, e questo è imposto dalla API, non dall'interfaccia.

Colonna	Tipo	Vincoli	Descrizione
id	BIGSERIAL	PK	Identificativo interno.
subject	VARCHAR(255)	NOT NULL, UNIQUE	Chiave naturale stabile: l'objectId di Entra quando il tenant lo rilascia, altrimenti <code>email:<indirizzo minuscolo></code> .
entra_object_id	VARCHAR(64)	UNIQUE parziale	Identificativo dell'oggetto in Entra ID, quando disponibile.
upn	VARCHAR(255)	—	User principal name. Non univoco e non identificante (vedi nota).
email	VARCHAR(255)	—	Indirizzo di posta. Assente per gli account amministrativi senza casella.
display_name	VARCHAR(255)	NOT NULL DEFAULT "	Nome mostrato in console.
given_name	VARCHAR(255)	—	Nome.
surname	VARCHAR(255)	—	Cognome.
role	VARCHAR(20)	NOT NULL DEFAULT 'viewer' CHECK	Uno fra admin, operator, viewer.
status	VARCHAR(20)	NOT NULL DEFAULT 'pending' CHECK	Uno fra pending, active, suspended.
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Primo accesso.
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Ultima modifica.
last_login_at	TIMESTAMP	—	Ultimo accesso riuscito.
profiled_at	TIMESTAMP	—	Quando è stato profilato.
profiled_by	VARCHAR(255)	—	Chi lo ha profilato.

Tabella 6 — Colonne della tabella `app\_users`.

INDICI

```
PRIMARY KEY (id)
UNIQUE      (subject)
UNIQUE INDEX uq_app_users_entra_object_id (entra_object_id) WHERE entra_object_id IS NOT NULL
INDEX idx_app_users_email_lower (LOWER(email))
INDEX idx_app_users_upn_lower   (LOWER(upn))
INDEX idx_app_users_status      (status)
```

Perché subject e non l'indirizzo di posta

Nessuno dei due funziona da solo: l'objectId può non essere rilasciato, e gli indirizzi di posta cambiano. `subject` è l'objectId quando c'è e un indirizzo normalizzato altrimenti, ed è ciò su cui si regge l'identità della riga.

Perché upn non è univoco

Lo UPN cambia a una rinomina o a uno spostamento di dominio. Usarlo come chiave farebbe ricadere ruolo e concessioni allo stato iniziale. Un vincolo di unicità sarebbe anche peggio: riassegnare un indirizzo a una persona nuova farebbe fallire il provisioning proprio per lei. La colonna esiste perché la convenzione che concede il ruolo di amministratore si valuta sullo UPN — gli account amministrativi non hanno casella, e valutata sulla posta la regola mancava esattamente gli account per cui esiste.

4.2 app_user_sedi

Le sedi concesse a un utente. È una tabella di associazione e non una colonna di tipo array, perché così la chiave esterna può fare il proprio lavoro: la cancellazione di una sede la fa sparire da ogni concessione, invece di lasciare un identificativo pendente che nulla può validare.

Colonna	Tipo	Vincoli	Descrizione
user_id	BIGINT	PK, FK → app_users(id) ON DELETE CASCADE	Utente.
sede_id	INTEGER	PK, FK → sedi(id) ON DELETE CASCADE	Sede concessa.
granted_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Momento della concessione.

Tabella 7 — Colonne della tabella `app\_user\_sedi`.

```
PRIMARY KEY (user_id, sede_id)
INDEX idx_app_user_sedi_sede_id (sede_id)
```

Le righe non sono significative per un amministratore: il ruolo `admin` comporta l'accesso a **tutte** le sedi a prescindere dalle concessioni registrate.

4.3 breakglass_users

Gli account di accesso locale di emergenza, usati solo quando Entra ID non è disponibile. Le righe sono create **esclusivamente** dalla riga di comando, mai via HTTP.

Colonna	Tipo	Vincoli	Descrizione
username	VARCHAR(100)	PK	Nome dell'account (per convenzione <code>bk.<nome></code>).
display_name	VARCHAR(255)	NOT NULL	Nome mostrato.
password_hash	TEXT	NOT NULL	Hash script auto-descrittivo: <code>script\$N\$r\$p\$salt\$hash</code> . Mai in chiaro.
role	VARCHAR(20)	NOT NULL DEFAULT 'admin'	Ruolo della sessione di emergenza. Il valore predefinito è <code>admin</code> perché questo account deve poter profilare i primi utenti.
enabled	BOOLEAN	NOT NULL DEFAULT TRUE	Abilitazione.

Colonna	Tipo	Vincoli	Descrizione
expires_at	TIMESTAMP	—	Scadenza dell'account. Raccomandata.
failed_attempts	INTEGER	NOT NULL DEFAULT 0	Tentativi falliti consecutivi.
locked_until	TIMESTAMP	—	Fine del blocco.
last_login_at	TIMESTAMP	—	Ultimo accesso riuscito.
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Creazione.
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Ultima modifica.

Tabella 8 — Colonne della tabella ``breakglass_users``.

Perché contatore e blocco stanno in banca dati

Se vivessero in memoria di processo, ogni replica della Container App avrebbe il proprio conteggio e il blocco sarebbe aggirabile semplicemente ritentando finché la richiesta non viene servita da un'altra replica. La limitazione per indirizzo di origine resta invece in memoria, per replica, ed è un controllo aggiuntivo, non quello principale.

La colonna `role` è aggiunta con un `ALTER TABLE` separato: `CREATE TABLE IF NOT EXISTS` non tocca una tabella già esistente, quindi su un database migrato la colonna può arrivare solo così. La validazione del valore è nel repository e non in un vincolo `CHECK`, perché non esiste un `ADD CONSTRAINT IF NOT EXISTS` su cui appoggiarsi.

4.4 session

Creata e gestita da `connect-pg-simple`, con `createTableIfMissing`. Contiene la sessione Express serializzata, compreso l'oggetto utente di passport e la sede corrente.

Colonna	Tipo	Vincoli	Descrizione
sid	VARCHAR	PK	Identificativo di sessione (la parte non firmata del cookie).
sess	JSON	NOT NULL	Contenuto della sessione: <code>passport.user</code> , <code>sedeId</code> , <code>wlcConnected</code> , <code>samlRedirect</code> .
expire	TIMESTAMP	NOT NULL	Scadenza, derivata da <code>cookie.expires</code> .

Tabella 9 — Colonne della tabella ``session``.

Il ruolo non è in sessione

La sessione contiene identità, non permessi. Un ruolo memorizzato qui sopravviverebbe fino alla scadenza del cookie — 24 ore — e un utente sospeso continuerebbe a operare per tutto quel tempo. L'autorizzazione è risolta a ogni richiesta a partire da `app_users` o da `breakglass_users`.

Una sessione di emergenza salva il cookie **due volte**: passport lo scrive con la durata predefinita di 24 ore e la rotta lo riscrive con la durata ridotta. Senza la seconda scrittura il browser smetterebbe di inviare il cookie alla scadenza breve, ma la riga resterebbe valida per 24 ore — e un cookie sottratto sopravviverebbe alla finestra che quell'impostazione deve imporre.

5 Tabelle di supporto

5.1 saml_request_ids

Gli identificativi delle AuthnRequest emesse e non ancora corrisposte, per la validazione `InResponseTo` contro il replay. Le righe sono minuscole e di vita breve: una per tentativo di accesso, eliminate oltre la finestra di validità (1800 secondi).

Colonna	Tipo	Vincoli	Descrizione
id	VARCHAR(255)	PK	Identificativo della AuthnRequest.
value	TEXT	NOT NULL	Valore associato dalla libreria.
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Emissione; la scadenza è calcolata in SQL.

Tabella 10 — Colonne della tabella `saml_request_ids`.

```
PRIMARY KEY (id)
INDEX idx_saml_request_ids_created_at (created_at)
```

La cache predefinita della libreria vive nella memoria del processo, e la sua stessa documentazione dichiara che «non sarà sufficiente» quando richiesta e risposta possono essere servite da processi diversi: su Container Apps è il caso normale. Un riavvio fra il reindirizzamento di accesso e la risposta dell'IdP basta a rompere la validazione; con più di una replica il problema diventa sistematico.

5.2 sms_config

Riga unica di configurazione del canale SMS. La funzione è presente a schema ma **dormiente**: il provider non è impostato e la funzionalità non è esposta nell'interfaccia.

Colonna	Tipo	Vincoli	Descrizione
id	INTEGER	PK DEFAULT 1	Riga unica.
gateway_type	VARCHAR(50)	—	Provider. Nullo: nessun provider configurato.
api_key	VARCHAR(255)	—	Chiave del provider. Oscurata nelle risposte della API.
sender_id	VARCHAR(11)	DEFAULT 'DompeGuest'	Mittente mostrato.
webhook_url	VARCHAR(500)	—	Endpoint di ritorno.

Tabella 11 — Colonne della tabella `sms_config`.

È l'unico segreto che risieda ancora in banca dati. Il campo è vuoto e la funzione nascosta; la direzione indicata è attivare il canale — spostando la chiave in Key Vault — oppure rimuoverlo.

5.3 wlc_config (legacy)

Residuo della modellazione precedente, quando i parametri del controller stavano in una tabella separata. È ancora **scritta** per compatibilità con le revisioni non ancora ritirate, ma **non è più letta** dal percorso operativo: `getWlcConfigBySede()` legge da `sedi`. La rimozione è prevista.

Colonna	Tipo	Vincoli	Descrizione
id	SERIAL	PK	Progressivo.
host, port, ssh_port	—	—	Parametri di connessione, ora su sedi .
username, wlan_ssid	—	—	Account e SSID, ora su sedi .
authenticated	BOOLEAN	DEFAULT FALSE	Colonna abbandonata: descriveva uno stato di sessione in una riga condivisa.
sede_id	INTEGER	UNIQUE parziale	Vecchio collegamento alla sede.

Tabella 12 — Colonne della tabella `wlc_config`.

Che cosa insegnava **authenticated**

Era una colonna condivisa da tutti gli operatori, scritta da un pulsante nell'intestazione della console, e la scrittura finiva sempre sulla prima riga qualunque fosse la sede in uso. Premere «Disconnetti» su una sede poteva quindi fermare in silenzio il provisioning di un'altra. Oggi lo stato della connessione è proprietà della sessione dell'operatore e non di una riga condivisa.

6 Vincoli e indici — riepilogo

Tabella	Chiave primaria	Unicità	Chiavi esterne	Altri indici
guests	id	username	—	status · created_at DESC · sede_id
sedi	id	code	—	code · active
app_users	id	subject · entra_object_id (parziale)	—	LOWER(email) · LOWER(upn) · status
app_user_sedi	(user_id, sede_id)	—	app_users(id) CASCADE · sedi(id) CASCADE	sede_id
breakglass_users	username	—	—	—
saml_request_ids	id	—	—	created_at
sync_logs	id	—	—	—
sms_config	id	—	—	—
wlc_config	id	sede_id (parziale)	—	—
session	sid	—	—	gestiti da connect- pg-simple

Tabella 13 — Riepilogo di vincoli e indici.

`guests.sede_id` non ha una chiave esterna verso `sedi`: la cancellazione di una sede è comunque rifiutata finché esistono ospiti collegati (`DELETE /api/admin/sedi/:id` verifica il conteggio), e la colonna può essere nulla per le righe anteriori all'introduzione delle sedi.

7 Migrazioni

7.1 Strategia

- Lo schema è definito in un unico blocco SQL in `backend/src/db/migrate.ts`, eseguito istruzione per istruzione.
- Ogni istruzione è **idempotente**: `CREATE TABLE IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`.
- Non esiste una tabella di versioni**: l'intero schema viene rigiocato a ogni esecuzione. È questo che rende l'idempotenza un requisito e non una comodità.
- Le migrazioni sono un passo della pipeline, non dell'avvio dell'applicazione: il job ACA dedicato gira **prima** che la nuova revisione entri in servizio.
- L'autenticazione del job è la stessa dell'applicazione: password nella stringa di connessione in sviluppo, token Entra ID in esercizio.

7.2 Esecuzione

```
# Job ACA (quello che esegue la pipeline)
az containerapp job start -n <migration-job> -g <rg>

# Manualmente, dentro il container backend
az containerapp exec -n <backend-app> -g <rg> \
  --command "node backend/dist/db/migrate.js"

# In sviluppo locale
npm run build -w backend && node backend/dist/db/migrate.js
```

Il comando esce con codice 0 in caso di successo e 1 in caso di errore, riportando l'istruzione fallita. In sviluppo, le migrazioni girano anche all'avvio dell'applicazione salvo `SKIP_MIGRATIONS=true`.

7.3 Il backfill dei parametri del controller

Una migrazione una tantum copia i parametri da `wlc_config` a `sedi`. La condizione `AND s.wlc_host IS NULL` è ciò che la rende sicura da lasciare in posizione: senza quella guardia, l'aggiornamento si ripeterebbe a ogni avvio e a ogni esecuzione del job, annullando in silenzio le modifiche appena fatte da un amministratore dal pannello.

```
UPDATE sedi s
  SET wlc_host = w.host, wlc_port = w.port, wlc_ssh_port = w.ssh_port,
      wlc_username = w.username, wlc_ssid = w.wlan_ssid
FROM wlc_config w
WHERE (w.sede_id = s.id OR w.id = s.wlc_config_id)
AND s.wlc_host IS NULL;
```

L'associazione è cercata su **entrambi** i lati del vecchio doppio collegamento, deliberatamente: i due potevano divergere, ed è proprio quella divergenza che il consolidamento elimina. Un fallimento del backfill non blocca il deploy — le colonne esistono comunque e il pannello può completarle — ma viene registrato come errore.

7.4 Bootstrap dell'account di emergenza

Al termine delle migrazioni, se `BREAKGLASS_SEED_PASSWORD` è valorizzata, viene creato l'account indicato da `BREAKGLASS_SEED_USERNAME` (per default `bk.guestportal`) se non esiste già. Serve a rendere

utilizzabile la directory: ogni utente SSO nasce bloccato, quindi qualcuno deve poter profilare i primi amministratori.

Nessuna password predefinita

Se la variabile è vuota l'account semplicemente **non viene creato**. Una password integrata nel codice sarebbe una porta di servizio pubblicata; una generata automaticamente sarebbe una password che nessuno conosce. Un fallimento del bootstrap non fa fallire il deploy, ma viene registrato come errore esplicito.

8 Dati di riferimento

La migrazione crea le tabelle **vuote**. Le cinque sedi sono dati anagrafici e vivono in `backend/src/db/seed.ts`. **Non esiste una API per crearle**: senza questo passo la prima schermata dopo l'accesso mostra «Nessuna sede configurata» e l'applicazione è inutilizzabile.

```
az containerapp exec -n <backend-app> -g <rg> \
  --command "node backend/dist/db/seed.js"

# output atteso: Reference data seeded successfully
```

code	name	city	wlc_host	wlc_ssid
MIL	Dompe Milano HQ	Milano	172.18.106.100	Dompe Guest
AQ	Dompe L'Aquila	L'Aquila	172.18.106.101	Dompe Guest AQ
NA	Dompe Napoli	Napoli	172.18.106.102	Dompe Guest NA
TIR	Dompe Tirana	Tirana	172.18.106.103	Dompe Guest TIR
SM	Dompe San Mateo	San Mateo	172.18.106.104	Dompe Guest SM

Tabella 14 — Dati di riferimento inseriti dal seed (porta HTTPS 443, porta SSH 22, account `admin_guest`).

Il seed inserisce, non aggiorna

Le sedi sono cercate per `code`: se la riga esiste, viene lasciata intatta. In passato il seed aggiornava nome, città e indirizzo a ogni esecuzione — innocuo finché quei valori esistevano solo nel codice, ma sono modificabili dal pannello di amministrazione e riapplicarli annullerebbe in silenzio la correzione di un amministratore al riavvio successivo. Per la stessa ragione `SEED_ENABLED` resta `false` in **produzione**: il popolamento si fa una volta con il comando qui sopra.

9 Dati personali e ritenzione

Dato	Dove	Note
Nome, azienda, referente dell'ospite	<code>guests</code>	Dati personali. Restano finché la riga non viene eliminata.
Indirizzo e telefono dell'ospite	<code>guests</code>	Dati personali di contatto. Il telefono è raccolto ma non utilizzato (canale SMS dormiente).
Indirizzo del destinatario	<code>sync_logs.url</code>	Compare nelle voci relative all'invio delle credenziali.
Nome, indirizzo e UPN dell'operatore	<code>app_users</code>	Provenienti dalle dichiarazioni SAML.
Identità in sessione	<code>session.sess</code>	Eliminata alla scadenza o all'uscita.
Password dell'ospite	—	Non persistita.
Password amministrativa del controller	—	Non persistita. Solo Key Vault.
Password di emergenza	<code>breakglass_users.password_hash</code>	Solo come hash script.

Tabella 15 — *Dati personali e sensibili presenti nello schema.*

CANCELLAZIONE

- Eliminare un ospite (`DELETE /api/guests/:id`) rimuove la riga e invia `no user-name` al controller.
- Eliminare un utente dalla directory ne rimuove anche le concessioni, per effetto della cascata. La voce viene ricreata — bloccata — al suo prossimo accesso.
- Le voci di `sync_logs` restano: conservano il nome utente dell'ospite, non il suo nome anagrafico. Una politica di ritenzione è raccomandata.

10 Accessi tipici e prestazioni

Operazione	Frequenza	Accesso	Indice utilizzato
Elenco ospiti della sede corrente	A ogni apertura e ogni 30 s per client	<code>SELECT ... FROM guests WHERE sede_id = ? [AND status = ?] ORDER BY created_at DESC</code>	<code>idx_guests_sede_id</code> , <code>idx_guests_created_at</code>
Timer	Ogni secondo	Lettura di tutti gli ospiti e aggiornamento di <code>elapsed_seconds</code> per quelli attivi	scansione — vedi nota
Sincronizzazione	Ogni 30 s per sede	Elenco sedi attive, elenco ospiti per sede, inserimenti e aggiornamenti mirati	<code>idx_sedi_active</code> , <code>idx_guests_sede_id</code>
Risoluzione dei permessi	A ogni richiesta, mitigata dalla cache	Lettura di <code>app_users</code> per <code>subject</code> e delle concessioni	unicità su <code>subject</code>
Lettura della sessione	A ogni richiesta	Lettura per <code>sid</code>	chiave primaria

Tabella 16 — Accessi tipici al database.

Il timer è il punto da sorvegliare

`tickTimers()` legge l'intero elenco degli ospiti una volta al secondo, senza filtro sullo stato. Con qualche migliaio di righe il costo resta trascurabile, ma è il primo punto che diventerà significativo se la tabella cresce: la mitigazione naturale è restringere la lettura agli ospiti `active` sfruttando `idx_guests_status`.